



RAG (Retrieval-Augmented Generation) Mastery R&D Guideline

≡ Tags

LLM

RAG

Vertex

RAG (Retrieval-Augmented Generation) Mastery R&D Guideline

Complete Step-by-Step Research, Actions & Milestones

Career Impact Summary – Why This Challenge Matters for Your AI Engineer Path

Main Goal Achievement in AI Engineer Career Path

Completing this guideline gives you **end-to-end production mastery** of Retrieval-Augmented Generation (RAG) systems. You will learn data preprocessing, embedding strategies, vector database selection (especially on GCP), advanced evaluation techniques (retrieval metrics + LLM-as-Judge), and how to build and deploy a full RAG pipeline. The result: you become a **production-ready RAG Engineer** who can solve real business problems with accurate, grounded, hallucination-resistant AI using open and cloud-native tools.

Why This Is Important

- **High Industry Demand:** In 2026, almost every company building LLM applications uses RAG for cost, accuracy, and data privacy. GCP-native RAG skills (Vertex AI + AlloyDB) are among the most requested for AI Engineer and GenAI Engineer roles in Vietnam and Southeast Asia.
- **Real Business Value:** RAG reduces hallucinations, enables private data usage, and delivers up-to-date answers — critical for customer support, internal search, and compliance-heavy industries.

- **Career Acceleration:** You move from “I can call an LLM” to “I can deliver reliable, measurable, production-grade RAG that scales on GCP.” This skill set drives promotions, higher salaries, and leadership roles in GenAI teams.
- **Efficiency Gains:** Proper chunking, embeddings, and vector DB choice can improve answer quality by 30–60% while keeping costs low.

Example of What You Will Do in a Real Project

After finishing this guideline, in a typical real-world project (e.g., building a **Household Energy Optimization & Sustainability Q&A Support System for Australian households**):

- Preprocess official Australian energy documents (rebates, solar incentives, energy tariffs, sustainability guidelines).
- Choose the right embedding model and vector database (e.g., AlloyDB pgvector).
- Evaluate retrieval with MRR/nDCG and use LLM-as-Judge for answer quality.
- Deploy the full RAG pipeline on Vertex AI as a scalable production system. Outcome: 50–70% reduction in hallucinations, accurate answers on cost-saving tips, solar optimization, government rebates, and sustainable home upgrades — helping Australian families save money while going green.

Step-by-Step Research, Actions & Milestones

Follow these steps in sequence. Each step includes **What to Research/Learn**, **Actions to Take**, and **Milestone/Deliverable**.

Step 1: Environment Setup & RAG Fundamentals

Research/Learn: RAG architecture (retriever + generator), LangChain/LlamaIndex/Haystack basics, GCP Vertex AI Generative AI and RAG Engine, HF datasets.

Actions: Set up GCP project (Vertex AI, Cloud Storage, Colab), create HF account, install core libraries.

Milestone: Successful test of a basic naive RAG pipeline (load document → embed → retrieve → generate).

Step 2: Data Preprocessing Techniques for RAG

Research/Learn: Chunking strategies (fixed-size, recursive character, semantic, sentence-based, hierarchical, page-level, LLM-based), cleaning (remove noise, metadata extraction), overlap strategies.

Actions: Experiment with different chunking methods on sample documents.

Milestone: Documented comparison of 3+ chunking strategies with pros/cons and a preprocessing pipeline ready for use.

Step 3: Embedding Methods – Research & Selection

Research/Learn: Popular models (Gemini Embedding 2, BGE-M3, Qwen3-Embedding, multilingual-e5-large, Jina v4/v5, Voyage AI, Cohere embed-v4); when to use each (multilingual, long documents, domain-specific); pros/cons (accuracy, latency, cost, self-hosting, context length).

Actions: Benchmark 3–4 models on sample data.

Milestone: Embedding model decision matrix (pros/cons table) + chosen model(s) for the POC.

Step 4: Vector Database Options for RAG

Research/Learn: GCP-native options (Vertex AI Vector Search, AlloyDB AI with pgvector + ScaNN index, Cloud SQL with pgvector) + others (MongoDB Atlas Vector Search, Qdrant, Weaviate, Pinecone, Chroma). When to use each (scale, hybrid SQL+vector, cost, ease of use), pros/cons.

Actions: Compare options in a decision table; test 2 GCP options locally/in Colab.

Milestone: Database selection guide with use-case recommendations and pros/cons documented.

Step 5: Building the Retriever Pipeline

Research/Learn: Indexing (embed + store), hybrid search (vector + keyword), reranking, metadata filtering.

Actions: Implement retriever using chosen embedding + database.

Milestone: Working retriever that returns top-K chunks for sample queries.

Step 6: Full RAG Pipeline Implementation

Research/Learn: Integration of retriever + LLM (Gemini on Vertex AI or open models), prompt engineering for grounded generation, handling context window

limits.

Actions: Build end-to-end RAG (query → retrieve → augment → generate).

Milestone: Functional RAG system answering questions on your test dataset.

Step 7: RAG Evaluation – Retrieval Metrics

Research/Learn: Curating test cases (ground-truth questions + relevant chunks), metrics (Precision@K, Recall@K, MRR, nDCG, Hit@K, MAP); when to use each, pros/cons.

Actions: Create a test dataset (20–50 questions) and compute all retrieval metrics.

Milestone: Evaluation notebook with retrieval metric results and interpretation.

Step 8: RAG Evaluation – LLM-as-Judge & End-to-End

Research/Learn: LLM-as-Judge technique (criteria: accuracy, completeness, relevance, faithfulness); frameworks like RAGAS, DeepEval, G-Eval; pros/cons vs human evaluation.

Actions: Implement LLM-as-Judge scoring on generated answers.

Milestone: Full evaluation report combining retrieval metrics + LLM-as-Judge scores.

Step 9: Advanced RAG Techniques

Research/Learn: Parent-child chunking, hierarchical indexing, query rewriting, agentic RAG, rerankers, feedback loops.

Actions: Apply 2–3 advanced techniques to improve baseline RAG.

Milestone: Improved RAG version with before/after evaluation results.

Step 10: Domain Adaptation & Australia-Specific Energy Research

Research/Learn: Australian household energy topics (solar PV, battery storage, energy tariffs, government rebates like STC, LGC, Victorian Energy Upgrades, NSW Peak Demand Reduction Scheme, sustainability standards, Clean Energy Regulator data, BOM weather data integration).

Actions: Collect and preprocess official Australian energy documents, policies, and public datasets relevant to household optimization and cost saving.

Milestone: Australia-specific knowledge base prepared and integrated into the RAG system.

Step 11: Model & Data Management on GCP & HF

Research/Learn: Storing embeddings/datasets on Cloud Storage, versioning on HF Hub, monitoring on Vertex AI.

Actions: Push processed datasets and models to HF Hub; set up GCP storage.

Milestone: Clean HF Organization + GCP bucket with all artifacts.

Step 12: Production Considerations on GCP

Research/Learn: Scaling, cost optimization, security, monitoring (Vertex AI RAG Engine), logging, compliance.

Actions: Review deployment patterns and best practices.

Milestone: Production readiness checklist documented.

Step 13: POC Project – Build & Deploy on GCP

Research/Learn: End-to-end integration on Vertex AI for a real-world use case.

Actions: Build the **Household Energy Optimization & Sustainability Q&A Support System for Australian Households**:

- Ingest official documents on energy tariffs, solar incentives, battery storage, rebates, sustainable home upgrades, and cost-saving tips.
 - Apply semantic/recursive chunking and chosen embeddings.
 - Use selected vector DB (e.g., AlloyDB pgvector or Vertex AI Vector Search).
 - Evaluate with full metrics + LLM-as-Judge.
 - Deploy as a Vertex AI endpoint or interactive Gradio/Streamlit app on Cloud Run for household users.
- Milestone:** Live POC RAG application on GCP with evaluation report and demo link.

Step 14: Final Integration & Documentation

Research/Learn: End-to-end best practices, cost analysis, versioning strategy.

Actions: Create complete report connecting all steps, including database/embedding choices, evaluation results, and POC outcomes.

Milestone: Final portfolio (GitHub repo + HF Organization + live GCP POC + full report).

Important Instruction for the Final Documents

During the entire process, actively research the latest developments (official GCP docs, HF blog, recent papers, Vertex AI updates). **If you discover anything missing** (e.g., newer chunking methods, emerging vector DB features, advanced LLM-as-Judge prompts, or better multilingual embeddings), **add it into the relevant sections of your final report and portfolio.**

If you see any improvements (better workflows, new tools, or optimizations), document them clearly and discuss recommendations with the team. This keeps the deliverable up-to-date and production-ready.

Final Success Criteria

- Working RAG pipeline with chosen preprocessing, embeddings, and database.
- Comprehensive evaluation using both retrieval metrics and LLM-as-Judge.
- Live POC (Australian Household Energy Optimization Q&A System) deployed on GCP with measurable results.
- Deep documented understanding of all topics (especially database options, embeddings, and evaluation).
- Final documents include any new discoveries or improvements found during research.